OpenHarmony-Audit-Agent 工具源代码技术说明书

OpenHarmony-Audit-Agent 面向 OpenHarmony 源码审计，将任务拆分、角色协作、反证检查、独立复核和证据记录组织为可恢复的工作流。本说明书介绍系统原理、关键模块、运行机制及工程验证结果。

1. 工具定位与技术路线

系统以角色化源码审计工作流为主体，覆盖目标拆分、源码调查、安全不变量提取、同类问题搜索、反证检查、候选冻结、独立盲验证和 PoC 计划。CodeQL-AI 提供按需辅助验证，用于核对特定路径、保护函数和查询模型。

审计线索可以来自源码阅读、人工定义的不变量、授权范围内的历史修复信息或静态告警。主体围绕线索建立调用链，说明攻击者可控输入、危险操作及约束关系。遇到可形式化的问题时，调用静态分析或局部符号验证，并将结果关联到候选版本。

大模型承担跨文件语义分析和任务组织。源码位置、输入快照、显式假设、反证记录与独立复核构成结论的依据；确定性验证用于进一步核对关键条件。各环节的中间产物均可保存，便于人工复核和任务恢复。

2. 工作流程与角色职责

总体流程为：目标快照与分片 → 源码调查与不变量提取 → 角色交接与反证 → 候选冻结 → 独立盲验证 → 历史核查与 PoC 计划 → 证据审核。辅助检查由候选中的具体问题触发，返回结果归入同一候选版本。

角色	主要职责
entrypoint-boundary	确认入口、攻击者能力、信任边界和可控参数
dataflow-lifetime	追踪数据流，核对容量、整数运算、对象生命周期和危险操作
counterevidence-history	查找调用方限制、权限检查、保护路径与授权范围内的历史依据
chain-composer	组织逐跳调用链，明确漏洞成立所需的不变量和条件
blind-verifier	使用独立上下文重建候选，形成独立判断
poc-planner	整理复现环境、触发条件、输入构造和观测点
reporter	汇总证据、影响范围、复核结论和修复建议

总控负责合并同根因项并发布候选版本。发现阶段与盲验证阶段通过任务包隔离上下文，历史查重在候选冻结及盲验证之后按策略执行。每个 worker 处理分配到的 analysis unit，共享验证资源按配置串行使用。检查点保存已完成事项、未解假设和下一步操作。

3. 目录与关键实现

模块	入口与主要行为
cli.py	统一命令入口；workflow 转发至 generic_cli；提供 validate-target、prepare、normalize-sarif 和 report
workflow/generic_contracts.py	解析工作流、目标和分析单元；检查标识、路径、阶段、角色与运行时参数
workflow/generic_adapters.py	提供目录、Git 仓库和反编译树接口；strict 模式检查源目录和单元存在性
workflow/generic_planner.py	按权重降序向当前负载最小的 worker 分配任务；生成计划、任务包、角色提示词与辅助策略
workflow/generic_runner.py	通过 ThreadPoolExecutor 调度 root workers；启动 Claude Code 或 Codex，保存事件日志并处理检查点与终态
workflow/checkpoint.py	读写 JSON 检查点；使用临时文件与 os.replace 原子替换，配合 worker-state 状态协议
workflow/scheduler.py	记录阶段事件；provider 调用与证据归档由调用层组织
contracts/models.py	定义 AuditCampaign、Finding、CandidatePacket、Evidence、Decision 和 ExperienceRecord；支持 JSON 序列化与内容哈希
evidence/normalizer.py	解析 SARIF 的 runs、results 和 codeFlows，保留规则、位置、路径与输入哈希
evidence/decision_gate.py	核对候选关联、源码或静态证据、调用链、不变量、反证和复核记录，输出四态结论
engines/codeql_ai/providers.py	封装经验检索、查询优化、函数读取和符号验证；通过 MCP 接口提供辅助能力
runtimes/	提供 Claude Code 与 OpenAI-compatible 独立事件适配器；主体 CLI 调度另由 generic_runner 实现
reporting/	从内置技术模板生成 Markdown 和 PDF；漏洞候选记录由独立数据契约表达

4. 运行时与任务恢复

workflow validate 检查配置及提示词资源，--strict-source 增加源目录和分析单元检查。workflow plan 生成确定性分片计划。workflow prepare 创建 worker 目录、任务包、角色提示词及共享候选、复核、报告目录。抽象目标可通过 --allow-missing-source 进行结构演练。

workflow run 读取已准备的任务包，根据 runtime.adapter 选择运行时。claude-code 使用 Claude Code 的 stream-json 输出；codex 使用 codex exec --json，以标准输入传递提示词，并设置 ephemeral 会话和 workspace-write 沙箱。command 指定可执行文件，model 指定模型名称。Codex 路径保留认证目录及必要系统环境；Claude Code 的隔离工作目录与凭据继承由配置控制。

runner 保存 controller 和 checkpoint 事件日志，并检查 worker-state.json 中的 terminal 字段和允许终态。未结束任务通过检查点会话整理状态，在 maxCycles 范围内继续；已处于终态的 worker 被跳过。workflow status 读取持久化状态。外部漏洞提交由人工处理。

OpenAI-compatible HTTP 适配器支持 base_url、模型名、API key 环境变量和流式事件，供独立调用。主体并行 runner 的运行时选项为 Claude Code 与 Codex CLI；HTTP 适配器的调用入口与 CLI 调度入口各自独立。

5. 候选与证据模型

CandidatePacket 包含稳定 ID、revision、源码快照、分析单元、位置、调用链、不变量、威胁模型、影响、反证记录和未解假设。候选冻结采用版本协议：实质修改生成新 revision，已有证据保留原版本归属。不可变存储由上层归档机制实现。

Evidence 记录 producer、candidate_id、source_ref、input_hash、content_hash、规则或模型版本、时间与执行状态。源码调查、静态告警、函数保护条件、符号结果、历史信息和 PoC 执行分别形成相应类型的证据。原始产物与哈希核验由调用方完成，reviewed 标志记录复核状态。

结论	判定条件
NEEDS_EVIDENCE	候选的基础证据或必要字段不足
STATIC_ONLY	具备源码审计或静态路径证据、位置、调用链、不变量、威胁模型及至少两条反证记录
CONFIRMED	基础条件成立，并具备独立复核标志和支持结论的确定性验证；验证记录的来源、哈希、快照、revision 与 reviewed 状态通过检查
REJECTED	存在已复核的决定性反证

证据审核函数检查记录结构和关联关系。证据真实性、路径覆盖和模型假设需要结合原始产物复核；模型置信度用于分析排序。低置信度经验作为调查线索，经过回归验证的经验按代码版本和适用条件使用。

6. CodeQL-AI 辅助验证

FP Experience 检索相似误报模式，帮助提出反证问题。Function-Level Sanitizer 搜索并读取路径上的保护函数。QL Optimizer 生成查询修补，修补结果通过重跑查询和正负例回归评估。Symbolic Sanitizer 在指定二进制、路径、分支选择与攻击谓词下开展局部可达性检查。

辅助调用固定候选版本、待核对问题和输入文件。返回结果需要记录执行状态、约束、覆盖范围及原始输出。SAT 表示约束内存在见证，UNSAT 表示所建模路径在给定约束下不可满足，UNKNOWN、错误和超时表示检查未获得确定结论。

当前实现提供 provider、MCP 入口及主体任务包中的辅助策略。provider 输出的 Evidence 规范化、复核与归档由调用层完成，自动闭环仍需集成。原理 HTML 通过交互展示这些接口之间的数据关系。

7. OpenHarmony 适配与报告结构

目标 profile 描述源码来源、tag 或 commit、模块路径、分析单元、构建信息和攻击者能力。多仓库目标使用版本清单固定快照。重点关注 IPC / Parcel、NAPI 回调、权限身份、缓冲区、整数与生命周期；硬件和系统服务验证通过目标专用 adapter 接入。

漏洞报告按源码位置与相关内容、CWE、公开 CVE 关联、触发条件、调用链、PoC 及运行环境、影响模块和版本、修复建议组织。候选状态与确定性验证结果一并保存，便于追踪结论形成过程。

8. 工程验证与运行实例

工程检查覆盖 Python 语法和导入、配置与 Schema、validate / plan / prepare / status、证据判定、运行时命令构造、状态恢复和离线页面交互。Codex CLI 已接入主体调度，并完成一组限定范围的 OpenHarmony Parcel 源码审计。

测试目标为官方 commonlibrary_c_utils 仓库，commit 为 b807d3dadb7dbb7639fe85b3a5f72da18d318da2。审计围绕 base/src/parcel.cpp 中的 WriteBuffer、EnsureWritableCapacity 和 WriteDataBytes，涉及四个直接辅助函数，并查阅 parcel.h。主文件共 1,686 行，审计范围按函数限定。

该次任务使用 Python 3.14.0、Codex CLI 0.163.0-alpha.10.2 和配置模型 gpt-6-astra，由单个 worker 顺序检查入口、数据流与保护条件。结果为 SOURCE_REVIEW_COMPLETE，worker 状态为 COMPLETE，候选数量为 0。产物包括 source-review.json、review-report.md、checkpoint.json 和 worker-state.json，保存在 reports/operation-demo/。本次运行覆盖源码审计及状态归档，独立盲验证、CodeQL、编译和 PoC 执行为单独验证环节。

9. 效果与性能评价

评价以主体工作流为基线，对比主体加按需辅助验证；单独 CodeQL 可作为外部参照。消融实验分别考察角色分工、反证和辅助检查。实验固定源码快照、输入线索、模型、预算和复核标准，统计候选准确性、已知基准召回、复核耗时和未解决比例。

性能按任务准备、主体审计、可选 CodeQL 建库与查询、辅助验证、报告阶段记录。现有测量包括三种规模的合成单元规划测试，以及上述 Parcel 单次限定函数审计。Parcel 运行墙钟时间为 129.60 秒，本机进程树采样峰值 RSS 为 385.45 MiB。详细环境、样本数量和统计口径见工具性能说明。

规划测试反映任务分配开销，Parcel 数据反映指定源码范围的单次审计成本。大规模全仓扫描、多 worker 扩展性和设备验证需要分别测量；模型语义判断与局部符号建模的结论适用于各自记录的假设和路径范围。

10. 来源与重构范围

主体来源：https://github.com/megakn1ght/audit-workflow-prompts ，commit 7bd9cd9cbea555980b347bfd71bd5e4e75087b9b。重构保留分片、角色协议、候选隔离、独立验证与 checkpoint，迁移入口至统一 CLI，并接入 Codex 调度。

辅助来源：https://github.com/Binnnn-Lab/CodeQL-AI ，commit 51dbf412f440f723746087588665646c0edadddd。保留必要源码、provider 和 MCP 入口，组织为主体按需调用的验证能力。统一数据契约、辅助调用策略、演示页面、说明文档和测量入口为本工程重构内容。来源归属及许可证信息见 NOTICE.md。